hashlock.

2025

Security Audit

Lazymog (Meme Token)

September 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Lazymog team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project’s team and community that the deployed contracts are secure.

Project Context

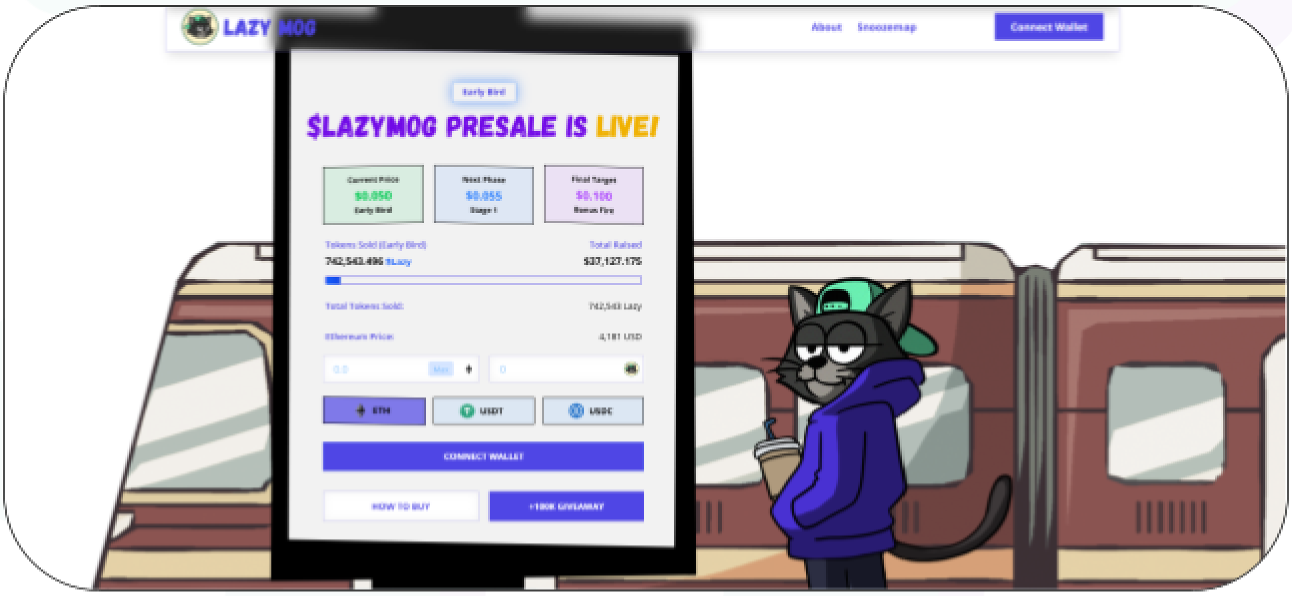
LazyMog is a meme coin project built on the Ethereum blockchain, embracing a playful anti-hustle culture. Its token, $LAZYMOG, promotes a laid-back lifestyle with slogans like "Zero Effort, Infinite Chill" and "Get Rich, Stay Asleep." The project offers a zero-tax tokenomics model and a presale with multiple stages, targeting community engagement through humor and a relaxed approach to cryptocurrency. LazyMog's roadmap, dubbed the "Snoozemap," includes plans for meme contests and community events, all while maintaining a focus on minimal effort and maximum enjoyment.

Project Name: Lazymog
Project Type: Meme Token
Compiler Version: ^0.8.20
Website: https://www.lazymog.com/
Logo:

Visualised Context:

Project Name	Lazymog
Launch Date	TBA
Compiler Version	v^0.8.20
Language	SOLIDITY
Network	ETHEREUM
Token Ticker	$LAZYMOG

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Lazymog project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Lazymog Smart Contracts
Platform	Ethereum / Solidity
Audit Date	September, 2025
Contract 1	LazyPresale.sol
Audited GitHub Commit Hash	25e997e398ceeef4dabcbceaa23505ec6d6e5309
Fix Review GitHub Commit Hash	acc688263525f48a8d9c609fe257874dc307ba8e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 High severity vulnerabilities

1 Medium severity vulnerability

1 Low severity vulnerability

1 Gas Optimisation

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
LazyPresale.sol - 11-stage presale with increasing prices - Accepts ETH, USDC, USDT with Chainlink price conversion - Fixed token allocation per stage prevents overselling - Tracks user contributions across all payment methods - Owner controls stage progression and emergency pause	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Lazymog project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Lazymog project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended

	solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] LazyPresale#getEthPrice - Insufficient oracle validation

Description

The `getEthPrice()` function lacks essential validations for oracle data freshness and reliability, making the contract vulnerable to stale prices.

Vulnerability Details

```
function getEthPrice() public view returns (uint256) {

    // Get latest price from Chainlink (returns price with 8 decimals)

    (, int256 price, , , ) = ethPriceFeed.latestRoundData();

    require(price > 0, "Invalid ETH price");

    // Convert to 18 decimals

    return uint256(price) * 10**10;

}
```

The function fails to validate `updatedAt` timestamp for data staleness and reasonable price bounds to detect oracle malfunctions.

Impact

Attackers can exploit stale oracle data during high volatility periods to purchase tokens at incorrect prices. In extreme cases, if oracle data becomes stale for hours, users could buy tokens significantly below or above market rates, causing financial losses to either the project or buyers.

Recommendation

Add a tolerance that compares the `updatedAt` return timestamp from `latestRoundData()` with the current `block.timestamp` and ensure that the price feed is being updated with the required frequency.

Status

Resolved

[H-02] LazyPresale#buyWithETH - Missing Slippage Protection

Description

The `buyWithETH()` function lacks slippage protection, exposing users to unfavorable token amounts due to ETH price volatility between transaction submission and execution.

Vulnerability Details

When users submit ETH purchase transactions, the ETH/USD price may change significantly before the transaction is executed, resulting in fewer tokens than expected. The current implementation provides no mechanism for users to specify minimum acceptable token amounts.

Impact

Users may receive fewer tokens than anticipated, especially during high market volatility. This creates poor user experience and potential financial losses for buyers who expected specific token amounts based on prices at transaction submission time.

Recommendation

Add a minimum token parameter (e.g. `minTokensOut`) to `buyWithETH` and require that `baseTokens >= minTokensOut`.

Status

Resolved

Medium

[M-01] LazyPresale#function - No Maximum Purchase Limits

Description

The contract lacks per-user or per-transaction purchase limits, allowing unlimited token concentration by individual addresses.

Vulnerability Details

Any user can purchase the entire allocation of a stage in a single transaction, leading to unfair token distribution and potential market manipulation. Large holders could control significant portions of the token supply from the presale phase.

Impact

Concentrated ownership can lead to market manipulation, reduced decentralization, and unfair distribution that contradicts typical presale objectives of broad community participation. Whale dominance may also negatively impact secondary market liquidity and price stability.

Recommendation

Implement reasonable purchase limits.

Status

Resolved

Low

[L-01] Ownable#transferOwnership - Consider Two-Step Ownership Transfer

Description

In OpenZeppelin's Ownable contract, ownership is transferred within one function call, namely `transferOwnership`. This pattern comes with its risks; any mistake can lead to losing the ability to access critical functionalities like pausing of the contract.

```
function transferOwnership(address newOwner) public virtual onlyOwner {

    if (newOwner == address(0)) {

        revert OwnableInvalidOwner(address(0));

    }

    _transferOwnership(newOwner);

}
```

Recommendation

Consider using Ownable2Step from the same repository for a safer ownership transfer. This prevents unintended changes and ensures the designated recipient is aware and capable of assuming ownership responsibilities.

Status

Resolved

Gas

[G-01] LazyPresale#buyWithETH, buyWithUSDC, buyWithUSDT - Redundant Assignment

Description

Redundant variable assignment wastes gas in purchase functions.

```
// Instead of:
uint256 baseTokens =  (ethInUSD * PRECISION)  / stage.price;
uint256 totalTokens = baseTokens;
// Use directly:
uint256 totalTokens =  (ethInUSD * PRECISION)  / stage.price;
```

Recommendation

Remove redundant assignments.

Status

Resolved

QA

[Q-01] LazyPresale - Missing Event Emission

Description

Critical functions `setToken()` and `withdrawERC20()` don't emit events, reducing transparency and monitoring capabilities.

Recommendation

Add appropriate events: `TokenSet` and `ERC20Withdrawn`.

Status

Resolved

[Q-02] LazyPresale - Event Parameter Indexing

Description

The `Contribution` event could benefit from indexing the `stageId` parameter for better event filtering capabilities.

Recommendation

```
event Contribution(address indexed user, address indexed token, uint256 amount, uint256 tokensAllocated, uint8 indexed stageId, uint256 timestamp);
```

Status

Resolved

Centralisation

The Lazymog project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Lazymog project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:
In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:
Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au